

树上路径问题

主讲：黄凤林

树上路径问题

✧ **树上路径问题**：就是不停询问树上点u到点v之间的边权极值、路径求和、最短路径等问题，同时路径修改后再次询问等，与区间问题类似。

✧ 如下图：不停询问点对之间的最短路径

2---7

3---6

3---4

5---6

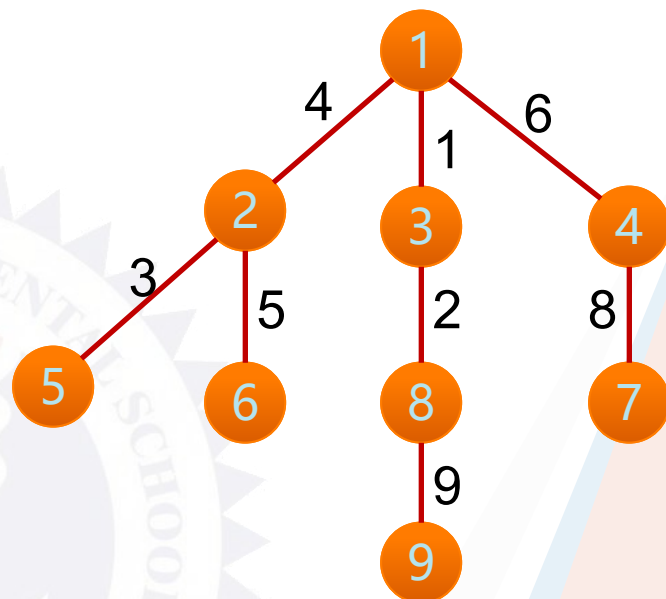


任意两点的最近公共祖先

→ 最近公共祖先

✧ 什么是最近公共祖先？

在一棵树上，每个节点肯定有其父亲节点和祖先节点，而最近公共祖先，就是两个节点在这棵树上**深度最大的公共的祖先节点**。换句话说，就是两个点在这棵树上**距离最近的公共相同祖先节点**。所以LCA主要是用来处理当两个点仅有**唯一一条确定的最短路径**时的路径。





→ 路径问题分析

1. 简单路径求极值，求和。

最近公共祖先

路径极值：只能用树上倍增算法

路径求和：可以用RMQ或者tarjan算法

2. 边更新、边求极值，求和。

树链剖分中的点剖、边剖（应用线段树的知识）

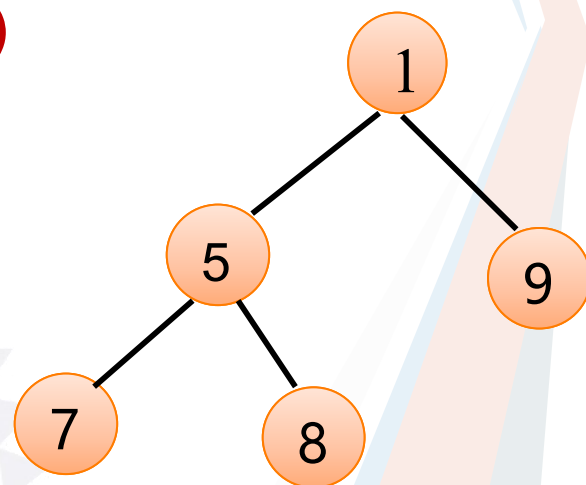
→ RMQ转LCA

- ✧ 考虑一个长度为 N 的序列 A ，按照如下方法将其递归建立为一棵树：
 - ① 设序列中最小值为 A_k ，建立优先级为 A_k 的根节点 T_k ；
 - ② 将 $A(1 \dots k-1)$ 递归建树作为 T_k 的左子树；
 - ③ 将 $A(k+1 \dots N)$ 递归建树作为 T_k 的右子树；
- ✧ 不难发现，这棵树是一棵优先级树

RMQ转LCA

我们以A序列为例建立树T: $A = (7\ 5\ 8\ 1\ 9)$

- 1、将最小元素1作为根节点左右分别建树
- 2、将最小元素5作为根节点左右分别建树



对于RMQ(A, i, j):

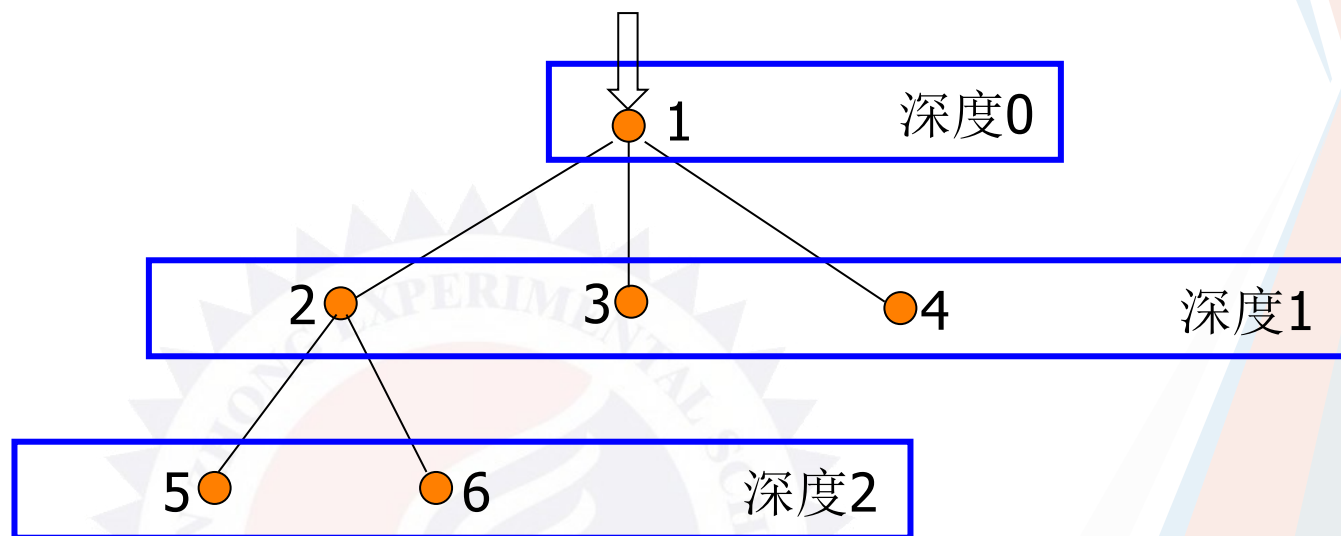
- ① 设序列中最小值为 A_k , 若 $k \in [i, j]$, 那么答案为 k ;
- ② 若 $k > j$, 那么答案为RMQ(A_{1..k-1}, i, j);
- ③ 若 $k < i$, 那么答案为RMQ(A_{k+1..N}, i, j);
- ④ 不难发现 $\text{RMQ}(A, i, j) = \text{LCA}(T, i, j)$!

这就证明了RMQ问题可以转化为LCA问题

→ LCA向RMQ的转化

- ✧ 对有根树T进行DFS，将遍历到的结点按照顺序记下，我们将得到一个长度为 $2N - 1$ 的序列，称之为T的欧拉序列F
- ✧ 每个结点都在欧拉序列中出现，我们记录结点u在欧拉序列中第一次出现的位置为 $\text{pos}(u)$

→ LCA向RMQ的转化



欧拉序列F: 1 2 5 2 6 2 1 3 1 4 1

深度序列B: 0 1 2 1 2 1 0 1 0 1 0

Pos (u): 1 2 8 10 3 5

→ LCA向RMQ的转化

- ✧ 根据DFS的性质，对于两结点 u 、 v ，从 $\text{pos}(u)$ 遍历到 $\text{pos}(v)$ 的过程中经过 $\text{LCA}(u, v)$ 有且仅有一次，且深度是深度序列 $B[\text{pos}(u) \dots \text{pos}(v)]$ 中最小的
- ✧ 即 $\text{LCA}(T, u, v) = \text{RMQ}(B, \text{pos}(u), \text{pos}(v))$ ，并且问题规模仍然是 $O(N)$ 的

→ LCA向RMQ的转化

遍历，结果保存在数组中

数组下标:	1	2	3	4	5	6	7	8	9	10	11	12	13
遍历序列:	A	B	D	B	E	F	E	G	E	B	A	C	A
节点在树中的深度:	1	2	3	2	3	4	3	4	3	2	1	2	1

要查询D和G的LCA:

- 1.在遍历序列中找到D和G第一次出现的位置, $\text{first}[D] = 3$, $\text{first}[G] = 8$ (3,8指数组下标)
- 2.取节点数组的[3,8]的那段序列, 查询一个最小值
 $\langle 3, 2, 3, 4, 3, 4 \rangle$, 最小值为2, 对应的节点是B, 所以D,G的LCA是B
- 3.用ST算法可以查询到最小值, 但是仅仅是知道最小值的值是多少, 并不知道最小值在哪

ST算法本质是DP, 在预处理构建DP数组的时候, 再用另一个数组pos数组记录位置

$\text{dp}[i][j]$: 表示从下标*i*开始长度为 2^j 的那段序列中的最小值

$\text{pos}[i][j]$: 表示对应 $\text{dp}[i][j]$ 状态的那个最小值的数组下标

例如 $\langle 2, 1, 2, 3, 4, 5 \rangle$ $\text{dp}[1][2]$ 表示从下标1开始到下标4里面的最小值

$\text{dp}[1][2] = 1$, $\text{pos}[1][2] = 2$, 表示1在下标2的位置

怎么得到pos数组

看看dp的转移方程

$\text{dp}[i][j] = \min \{ \text{dp}[i][j-1], \text{dp}[i+2^{j-1}][j-1] \}$

If $\text{dp}[i][j-1] < \text{dp}[i+2^{j-1}][j-1]$, $\text{pos}[i][j] = \text{pos}[i][j-1]$;

else $\text{pos}[i][j] = \text{pos}[i+2^{j-1}][j-1]$;

→ LCA向RMQ的转化

思考:记录位置一定要用pos数组吗, 不是的, 可以省略掉pos数组

- 1.首先我们DP找的最小值, 其实是节点深度, 即深度序列里面的内容, 而深度序列是已经保存下来的了, 上面的DP方法, 我们只在初始化DP数组的时候用到了它, 那为什么不直接用DP数组来记录位置, 然后取出位置, 直接用深度序列数组来比较呢?

说得有点复杂, 看看下面的代码

```
void ST(int len)
{
    int K = (int)(log((double)len) / log(2.0));
    for(int i=1; i<=len; i++) dp[i][0] = i;
    for(int j=1; j<=K; j++)
        for(int i=1; i+_pow[j]-1<=len; i++)
        {
            int a = dp[i][j-1], b = dp[i+_pow[j-1]][j-1];
            if(R[a] < R[b]) dp[i][j] = a;
            else dp[i][j] = b;
        }
}

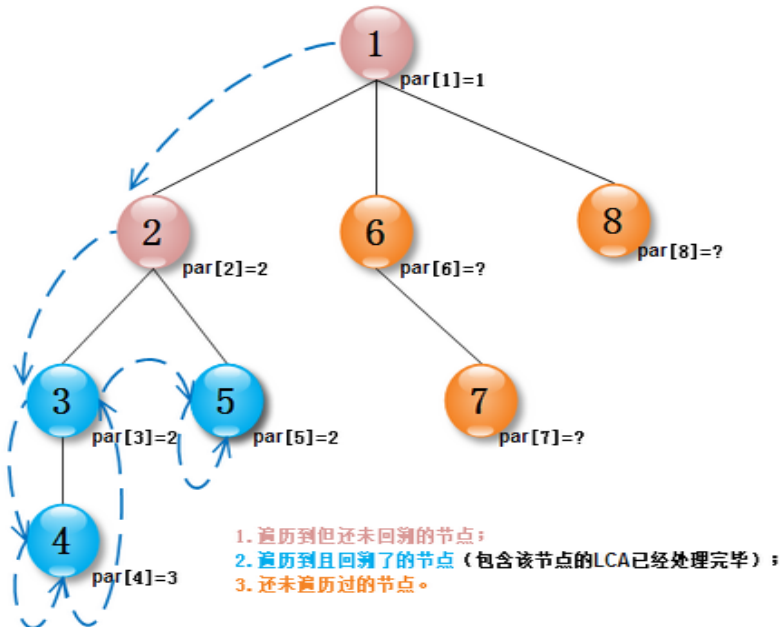
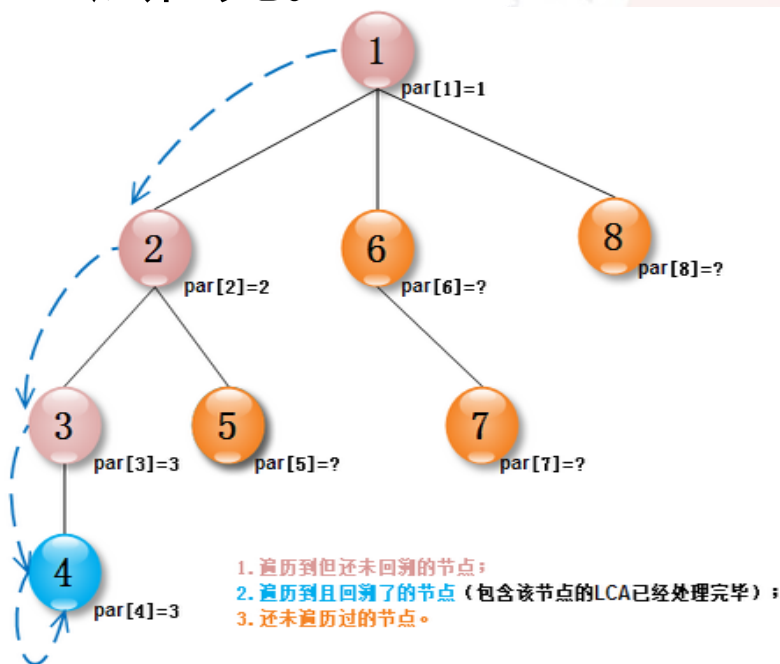
int RMQ(int x, int y)
{
    int K = (int)(log((double)(y-x+1)) / log(2.0));
    int a = dp[x][K], b = dp[y-_pow[K]+1][K];
    if(R[a] < R[b]) return a;
    else return b;
}
```

学习参考博客:

<http://www.cnblogs.com/scau20110726/archive/2013/05/26/3100812.html>

→ Tarjan求LCA

- ✧ Tarjan求解LCA算法是一个**离线的算法**。即根据输入的询问对来dfs处理与之对应的LCA。
- ✧ 解题思路：利用dfs+并查集解决，利用dfs在递归回溯过程中，处理完该点子树的所有最近公共祖先。非子树内的节点与子树内节点的最近公共祖先一定是其子树内的最上层祖先。



→ Tarjan求LCA

解题步骤:

- 1、从树的根节点(root)开始深搜
 - 2对于新搜索到的一个节点(x)，首先创建由这个结点构成的集合(由father[]数组维护，father[x]=x，当前这个集合中只有元素x)
 - 3、然后依次搜索并处理该节点包含的所有子树（搜索和处理是个递归的过程。
- 结合第5点理解：每搜索完一棵子树，则可确定子树内的LCA询问都已解，
其他的LCA询问的结果必然在这个子树之外。)

学习参考博客：<http://www.cnblogs.com/JVxie/p/4854719.html>

→ Tarjan求LCA

```

void tarjan_lca(int x){
    father[x]=x;//构建以x为根的集合
    visit[x]=true;
    for(int i=head[x];i>0;i=e[i].next) { //枚举相邻边，递归子树
        if(!visit[e[i].to])//如果当前节点没有访问过
        {
            tarjan_lca(e[i].to);//递归子树
            father[e[i].to]=x;//回溯集合合并子树
        }
    }
    for(int i=head_q[x];i>0;i=e_q[i].next)//处理包含x点对询问且以遍历最近公共祖先
    {
        if(visit[e_q[i].to]) //如何当前询问对的节点遍历过
        {
            int lca=find(e_q[i].to);
            printf("%d-->%d: %d\n",x,e_q[i].to,lca);
        }
    }
}

```

➔ 求最近公共祖先

什么是最近公共祖先？

回顾RMQ转最近公共祖先的求法：

- (1)、dfs求出树的欧拉序列 $E[i]$,并记录下与之对应的节点深度 $B[i]$
- (2)、RMQ预处理 $B[i]$ 数组。 $dp[i][j]$
- (3)、根据询问不停返回 $u \rightarrow v$ 在 B 中的最小值。与之对应 E 中就是最近的公共祖先。

练习题目：校门外的树

➔ RQM转LCA的弊端

优势：算法思想简单，很容易快速求出 $u-v$ 的最近公共祖先节点

缺点：不能够求解 $u-v$ 路径上权值最小值或最大值问题。如2013 noip day1-3。

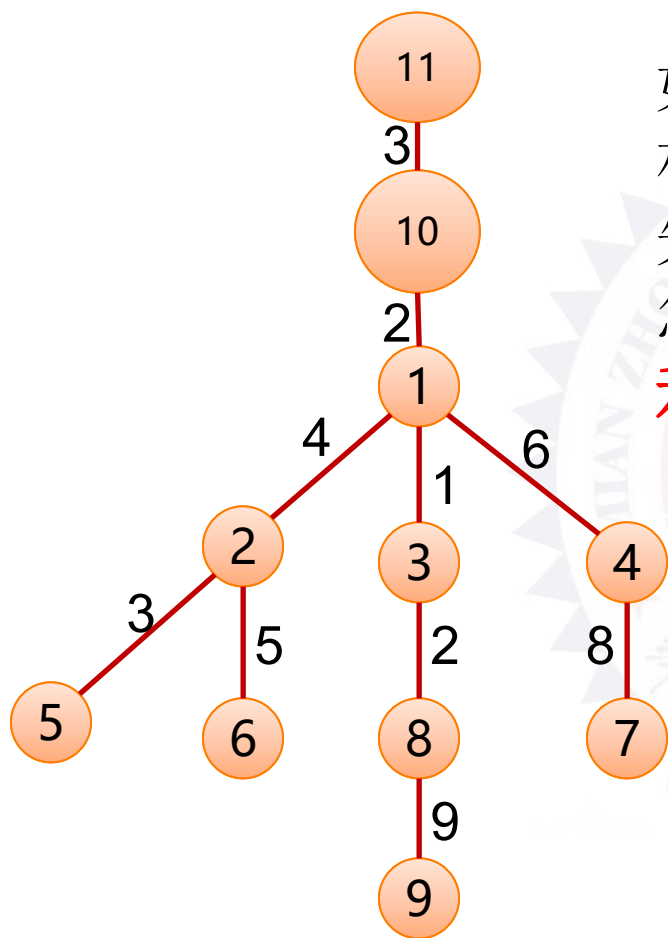
如何才能快速求出LCA且路径权值的最值问题呢？

树上倍增



树上倍增

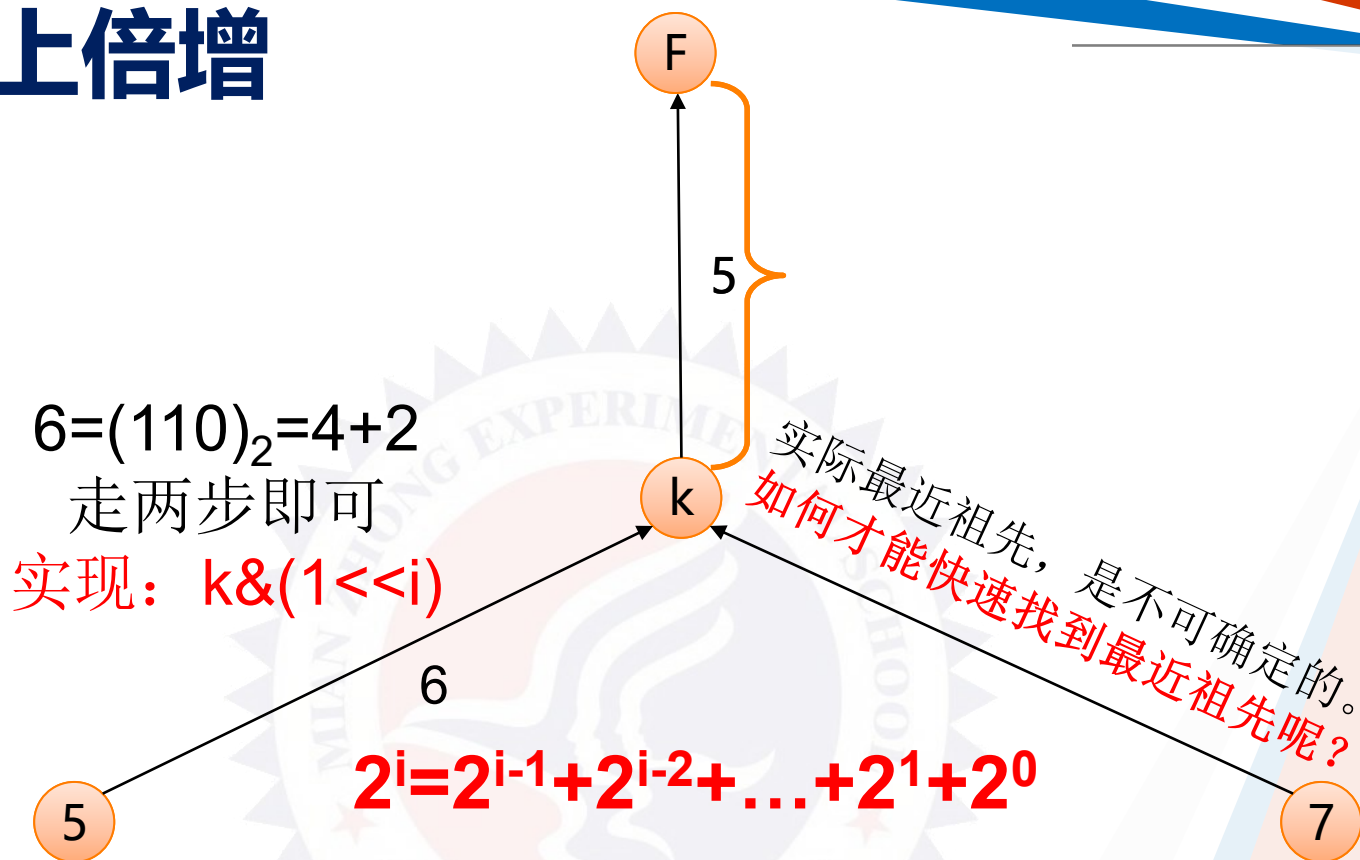
如何快速找到*i*点和*j*点的公共祖先



如何快速找到5和7的最近公共祖先？
朴素：从5和7开始依次向上遍历，最先找到的相同节点即为最近公共祖先。
怎么样优化朴素方法呢？

利用倍增思想，快速遍历

树上倍增



注：当所求两点不在同一高度时？
先调整两点到统一高度，再向上倍增

➔ 倍增算法流程

- 1、dfs预处理出i点向上 2^j 的节点
- 2、倍增遍历，快速求出i和j点的最近公共祖先。
- 3、根据需求，倍增查询i和j到祖先间的值。

树上倍增思想：就是通过RMQ的思想，快速遍历i点的祖先。
于是采用RMQ的思想对u点的祖先预处理。

用 $fa[i][j]$ 表示i点的第 2^j 个祖先，快速实现上层祖先的查找，
转移方程： $fa[i][j] = fa[fa[i][j-1]][j-1]$

$dp_min[i][j]$ 表示从i点到第 2^j 个祖先的最小值，
转移方程： $dp_min[i][j] = \min(dp[i][j-1], dp[fa[i][j-1]][j-1])$



Dfs预处理

```
void dfs(int x)
{
    visit[x]=true;
    for(int i=1;(1<<i)<=deep[x];i++)//这里在完成x点到它上层祖先点的预处理
    {
        fa[x][i]=fa[fa[x][i-1]][i-1];
        dp_min[x][i]=min(dp_min[x][i-1],dp_min[fa[x][i-1]][i-1]);
    }
    for(int i=head[x];i>0;i=edge[i].next)
    {
        if(!visit[edge[i].to])
        {
            fa[edge[i].to][0]=x;
            dp_min[edge[i].to][0]=edge[i].w;
            deep[edge[i].to]=deep[x]+1;
            dfs(edge[i].to);
        }
    }
}
```

→ 求LCA

```
int lca(int x,int y)
{
    if(deep[x]<deep[y]) swap(x,y);
    int t=deep[x]-deep[y];// 计算深度差
    for(int i=0;(1<<i)<=t;i++) if(t&(1<<i)) x=fa[x][i];// 深度差调整到同一高度
    if(x==y) return x;// x和y在一条链上
    for(int i=17;i>=0;i--)// 向上枚举找最近公共祖先，从根开始向下，思考为什么？
    {
        if(fa[x][i]!=fa[y][i])
        {
            x=fa[x][i];y=fa[y][i];
        }
    }
    return fa[x][0];
}
```



→ 查询

```
int ask(int x,int f)
{
    int mm=0x7f7f7f;
    int t=deep[x]-deep[f];
    for(int i=0;(1<<i)<=t;i++)
    {
        if(t&(1<<i))
        {
            mm=min(mm,dp_min[x][i]);
            x=fa[x][i];
        }
    }
    return mm;
}
```

→ 树上倍增注意的几个问题

- 1、dfs过程中，dfs到v点，需要在v节点的深度范围内，表示完v节点 2^j 的祖先
- 2、给定u,v,如何快速找到最近公共祖先的一个转换。
- 3、找到u,v的祖先k后，如何快速求出u—k和v—k的最值。
- 4、核心用到了深度T与 2^j 的“&”运算，即

$$T \& (1 \ll j)$$



➔ 倍增必写练习题目

✧ 落谷：P3379

✧ 落谷：P2680

✧ 落谷：P1967 (noip 2013 day1-3)





→ 练习题目

- ✧ [hdu 2586 How far away ?](#)
- ✧ LCA模板题
- ✧ 题意：给一个无根树，有q个询问，每个询问两个点，问两点的距离。求出 $lca = LCA(X, Y)$ ，然后 $dir[x] + dir[y] - 2 * dir[lca]$
- ✧ $dir[u]$ 表示点u到树根的距离





→ 练习题目

✧ poj 1986 Distance Queries LCA

- ✧ 题意：LCA模板题，输入n和m，表示n个点m条边，下面m行是边的信息，两端点和权，后面的那个字母无视掉，没用的。接着k，下面k个询问lca，输出即可
- ✧ 有人说要考虑不连通的情况，我没考虑AC了，另外可能有u，u这样的询问，不过这不影响，照样是写模板，没有特判，一样能过



→ 练习题目

✧ [poj 3694 Network](#)

- ✧ 连通分量 + LCA：先缩点，再求LCA，并
- ✧ 且不断更新，这题用了朴素方法来找LCA，
- ✧ 并且在路径上做了一些操作

✧ [poj 3417 Network](#)

- ✧ LCA + Tree DP：在运行Tarjan处理完所有的LCA询问后，进行一次树DP，树DP不难，但是要想到用树DP并和这题结合还是有点难度



→ 练习题目

✧ [poj 3728 The merchant](#)

- ✧ LCA + 并查集的变形，优化：好题，难题，思维和代码实现都很有难度，需要很好地理解Tarjan算法中并查集的本质然后灵活变形，需要记录很多信息（有点dp的感觉）

✧ [hdu 3830 Checkers](#)

- ✧ LCA + 二分：好题，有一定思维难度。先建立出二叉树模型，然后将要查询的两个点调整到深度一致，然后二分LCA所在的深度，然后检验。



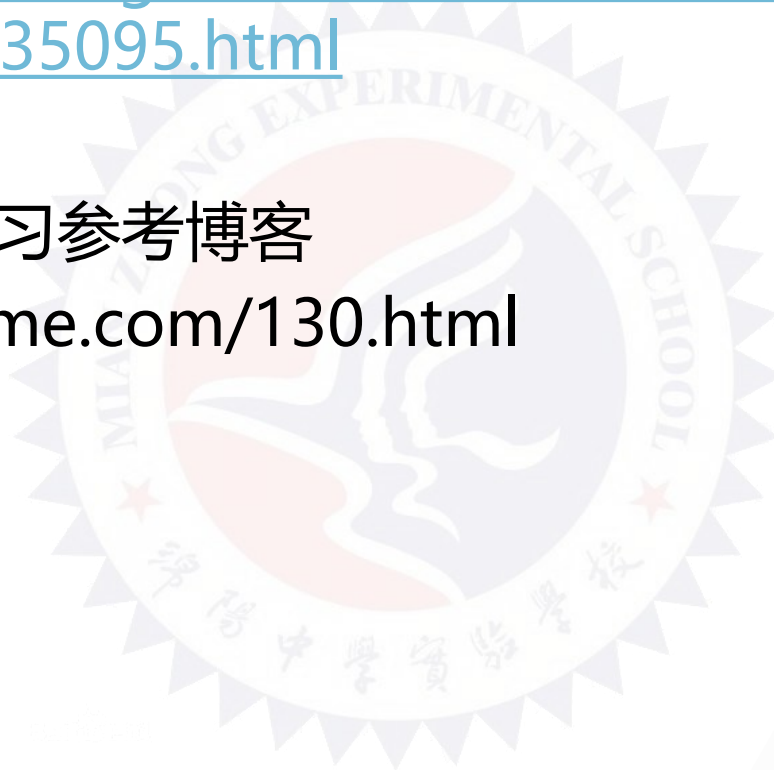
→ 练习题目

练习题目参考博客

<http://www.cnblogs.com/scau20110726/archive/2013/06/14/3135095.html>

Tarjan LCA 学习参考博客

<http://alanyume.com/130.html>





→ 树上差分

- ✧ 什么是差分？（差分序列）序列元素间相邻元素相减构成的新序列，叫做差分序列
- ✧ $1\ 3\ 5\ 8\ 18 \rightarrow 0\ 2\ 2\ 3\ 10$, 这就是差分序列,
- ✧ 差分序列作用：差分序列的前缀和再加上首元素的值，即该位置原来元素的值。
- ✧ 差分序列的应用：
 - 区间修改变为点修改
 - 统计位置被覆盖和修改的次数等。
 - 公共序列等



→ 例题

- ✧ 在直角坐标系上，给定 n 条若干长度的线段覆盖坐标系，求被覆盖次数最多的次数，及最多次数下的覆盖总长度。
- ✧ 注：一条线段覆盖可能为一个点。
- ✧ 数据范围： $N \leq 10^5$ ，线段长度 $\leq 10^5$

→ 例题

✧ 问题分析：

✧ 朴素：把直角坐标系离散成一个一个的整数点 $a[i]$ 表示，然后每条线段 $[x,y]$ 覆盖，则修改 $[x,y]$ 的值，最后统计 $a[i]$ 上的值，求出最大值及其长度即可。
预估分数10—30

✧ 优化：

- ✧ 1、线段树、树状数组各种高级数据结构,期望得分100分，但代码实现能力要求高。
- ✧ 2、序列差分，预估得分100分,So easy!

➔ 差分做法

- ✧ 差分做法：
- ✧ 把线段覆盖，改成端点处理，针对 $[x,y]$ 线段覆盖，则把 $a[x]++$, $a[y+1]--$ 即可，最后统计序列 $a[]$ 中最大前缀和，即为覆盖次数最多。
- ✧ 距离如下：
- ✧ 覆盖线段： $[1,3]$ 、 $[2,5]$, $[3,8]$
- ✧ 则 a 序列值为：

1	2	3	4	5	6	7	8	9
1	1	1	-1	0	-1	0	0	-1



→ 参考代码

```
const int maxn=100000+10;
int a[maxn];
int maxm=0,sum[maxn];
int ans=0;
for(int i=1;i<=n;i++)
{
    scanf("%d%d",&x,&y);
    a[x]++,a[y+1]--;
}
for(int i=1;i<=n;i++)
{
    sum[i]+=sum[i-1]+a[i];
    maxm=max(maxm,sum[i]);
}
for(int i=1;i<=n;i++){
    if(sum[i]==maxm) ans++;
}
printf("%d\n",ans);
```





➔ 练习题

✧ 问题描述

有这样一道题目：给你一个 $m \times n$ 的矩阵，然后使用 k 块地毯铺地。每片地毯都给出左下角和右上角坐标。问所有地毯铺完之后，还有多少个整点（所谓整点，即横、纵坐标均为整数的点）没有被地毯覆盖。

✧ 数据范围： $n, m \leq 100000$



→ 树上差分

✧ 在讲树上差分之前，首先需要知道树的以下两个性质：

- (1) 任意两个节点之间有且只有一条路径。
- (2) 一个节点只有一个父亲节点。

假设我们要考虑的是从 u 到 v 的路径， u 与 v 的lca是 a ，那么很明显，如果路径中有一点 u' 已经被访问了，且 $u' \neq a$ ，那么 u' 的父亲也一定会被访问，这是根据以上性质可以推出的。

所以，我们可以将路径拆分成两条链， $u \rightarrow a$ 和 $a \rightarrow v$ 。那么树上差分有两种常见形式：

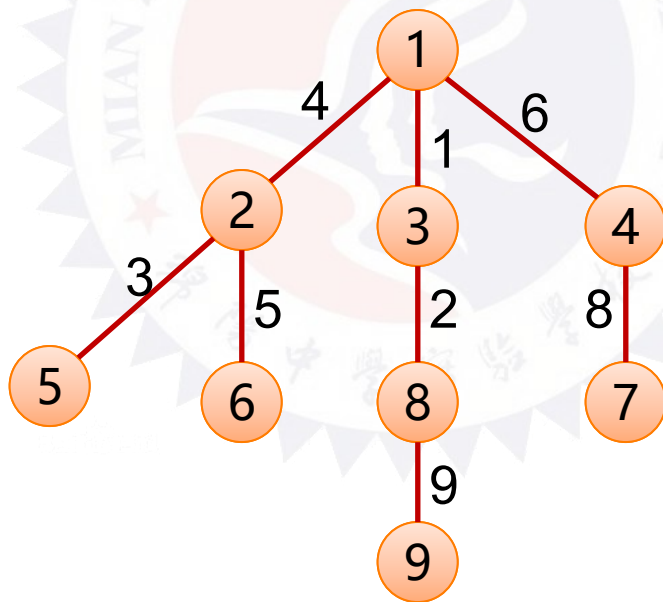
- (1) 关于边的差分；
- (2) 关于点的差分。

树上差分

关于边的差分:

将边拆成两条链之后，我们便可以像差分一样来找到路径了。因为关于边的差分， a 是不在其中的，所以考虑：

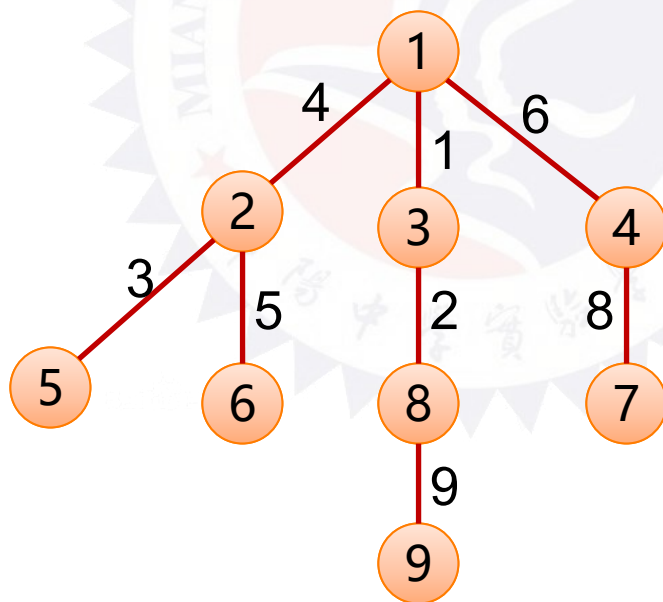
链 $u \rightarrow a$, 则使 $cf[u]++$, $cf[a]--$; 链 $a \rightarrow v$, 也是 $cf[v]++$, $cf[a]--$ 。
合起来是 **$cf[u]++$, $cf[v]++$, $cf[a]-=2$**



→ 树上差分

2、关于点的差分：

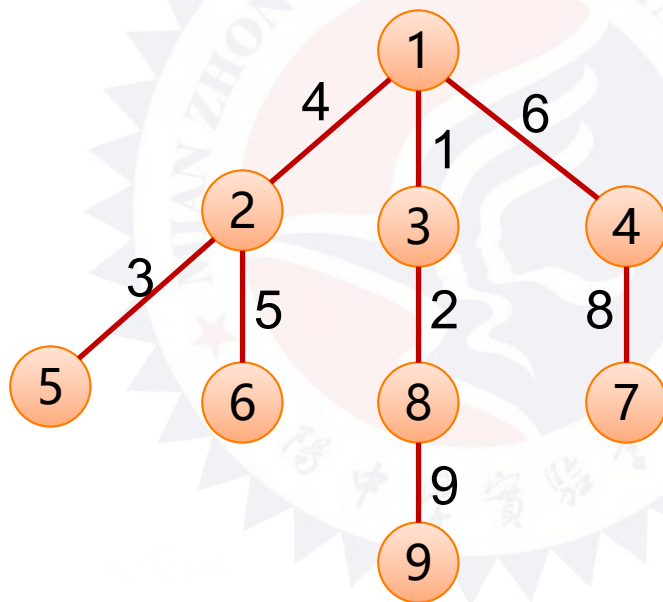
还是与和边的差分一样，对于所要求的路径，拆分成两条链。步骤也和上面一样，但是也有一些不同，因为关于点， u 与 v 的 lca 是需要包括进去的，所以要把 lca 包括在某一条链中，最后对 cf 数组的操作便是： **$cf[u]++$** , **$cf[v]++$** , **$cf[a]--$** , **$cf[father[a]]--$** 。其时间复杂度也是一样的 $O(n)$ 。



树上差分

树上差分的两种应用（两种操作）

- 已知路径求被所有路径覆盖的边
- 已知路径求树上所有节点被路径覆盖次数



→ 树上差分

✧ 已知路径求被所有路径覆盖的边

- 首先对已知的这 n 条路径的**起点 a** 和**终点 b** 的权值 $+1$,并对 $\text{lca}(a,b)$ 的权值 -2 。
- 从根节点开始深搜，回溯时将其本身的权值加上所有子节点的权值。
- 那么满足要求的边就是**权值等于 n 的节点与其父节点所连的边**。

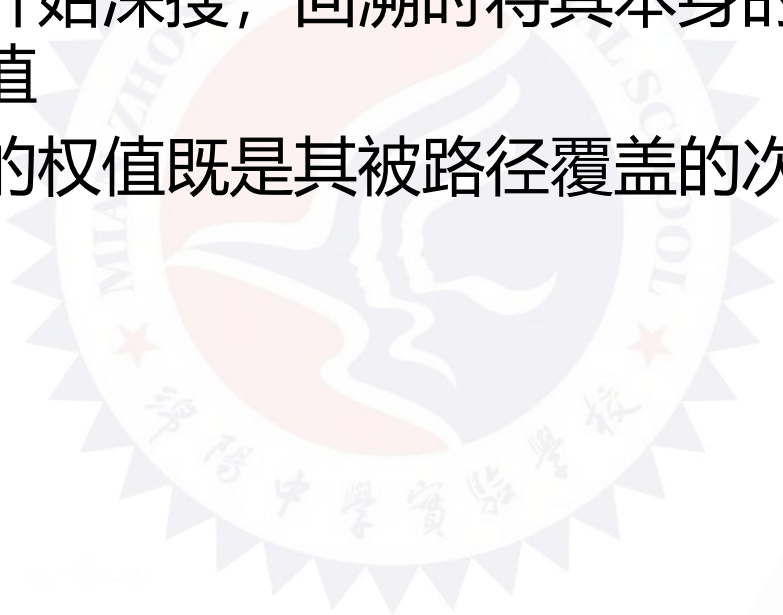
```
void dfs(int x, int father){ //x为深搜的节点, father 为 x 节点的父节点
    int v;
    for (int i=head[x]; i; i=edge[i].next)
    {
        v=edges[i].to; //枚举由x节点能够到达的所有的节点
        if (v != father) { //筛掉x节点的父节点
            dfs(v, x); //深搜to节点, 并设置其父节点为x
            cf[x] += cf[v]; //累加权值和
        }
    }
}
```



→ 树上差分

✧ 已知路径求树上所有节点被路径覆盖次数

- 对每条路径的起点 a 和终点 b 的权值 $+1$ ，对 $\text{lca}(a,b)$ 的权值 -1 ，对 $\text{lca}(a,b)$ 的父节点权值 -1
- 从根节点开始深搜，回溯时将其本身的权值加上所有子节点的权值
- 每个节点的权值既是其被路径覆盖的次数





树上差分应用

P2680 运输计划

题目描述

L 国有 n 个星球,还有 $n-1$ 条双向航道,每条航道建立在两个星球之间,这 $n-1$ 条航道连通了 L 国的所有星球。

小 P 掌管一家物流公司,该公司有很多个运输计划,每个运输计划形如:有一艘物流飞船需要从 u_i 号星球沿最快的宇航路径飞行到 v_i 号星球去。显然,飞船驶过一条航道是需要时间的,对于航道 j ,任意飞船驶过它所花费的时间为 t_j ,并且任意两艘飞船之间不会产生任何干扰。

为了鼓励科技创新,L 国国王同意小 P 的物流公司参与 L 国的航道建设,即允许小 P 把某一条航道改造成虫洞,飞船驶过虫洞不消耗时间。

在虫洞的建设完成前小 P 的物流公司就预接了 m 个运输计划。在虫洞建设完成后,这 m 个运输计划会同时开始,所有飞船一起出发。当这 m 个运输计划都完成时,小 P 的物流公司的阶段性工作就完成了。

如果小 P 可以自由选择将哪一条航道改造成虫洞,试求出小 P 的物流公司完成阶段性工作所需要的最短时间是多少?



练习题目

P2680 运输计划

P3128最大流Max Flow



→ 树链剖分

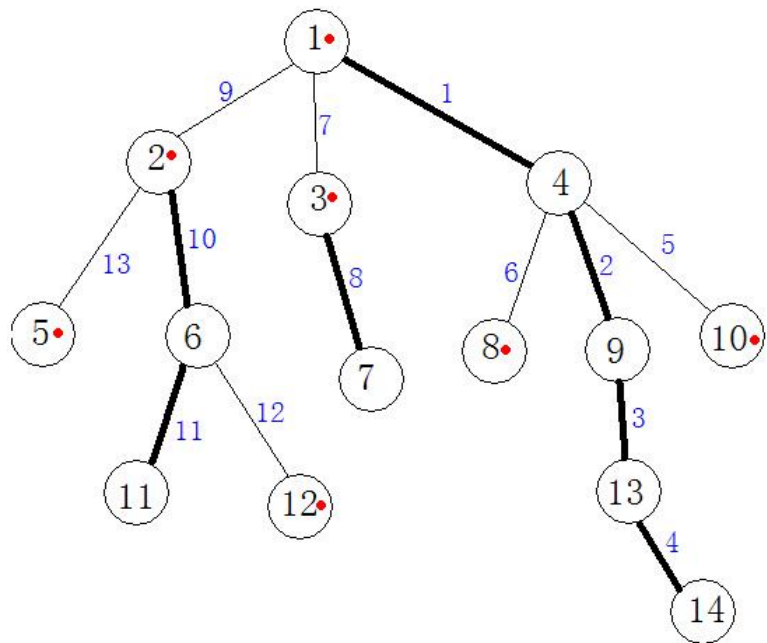
✧ 问题提出:

给定 n 个点的一棵树，每条边上有一个权值， m 次询问或者修改 $u \rightarrow v$ 路径上的值。针对每次询问，输出对应的答案

解决方法：之前学的几个方法，都不行，必须探索新的方法---**树链剖分**。

树链剖分的思想：就是把一棵树按照一定规则分解成若干条连续的链，再对路径进行预处理，将路径上的数据分为几段，然后再对这几段求最大值，有点像分桶法。

下面来看一个例证



比如我们要求解从顶点11到顶点14路径上边权的最大值，我们可以求出1-4-9-13-14这条路径上的最大值s1，然后求出1-2路径上的最大值s2,然后再求出2-6-11路径上的最大值s3。然后我们再求解 $\max(s1,s2,s3)$ 的值就可以得到结果

从图上我们可以看出，重儿子所在的路径是子树中路径最长的，这也是为什么重儿子的定义是子树中节点最多的子节点，然后我们也发现黑色的路径上的蓝色编号是连续的，这是为了将这个长的路径保存在一段连续的区间，那么我就可以了对这块连续区间进行操作了。

也许大家已经想到了这样规定的好处，因为越长的，我就一起处理，效率自然是越高了，重儿子就是这么规定而来的。

➔ 轻重链剖分

✧ 树链剖分的第一步当然是对树进行**轻重边的划分**。

定义 $\text{size}(x)$ 为以 x 为根的子树节点个数，令 v 为 u 的儿子中 size 值最大的节点，那么 (u,v) 就是重边，其余边为轻边， v 就是**重儿子**，其他相关儿子为轻儿子。

✧ 两个重要的性质：

(1) 轻边 (u,v) 中， $\text{size}(v) \leq \text{size}(u/2)$

(2) 从根到某一点的路径上，不超过 $\log n$ 条轻边和不超过 $\log n$ 条重路径。

➔ 树链剖分储备数组

siz[]数组，用来保存以x为根的子树节点个数

top[]数组，用来保存当前节点的所在链的顶端节点

son[]数组，用来保存重儿子

dep[]数组，用来保存当前节点的深度

fa[]数组，用来保存当前节点的父亲

tid[]数组，用来保存树中每个节点剖分后的新编号

pos[]数组，用来保存当前节点在线段树中的位置

➔ 树链剖分实现过程

1、我们可以用两个dfs来求出fa、dep、siz、son、top、pos

dfs_1: 把fa、dep、siz、son求出来

dfs_2:

1 对于v, 当son[v]存在 (即v不是叶子节点) 时, 显然有 $\text{top}[\text{son}[v]] = \text{top}[v]$ 。线段树中, v的重边应当在v的父边的后面, 记 $w[\text{son}[v]] = \text{totw} + 1$, totw表示最后加入的一条边在线段树中的位置。此时, 为了使一条重链各边在线段树中连续分布, 应当进行dfs_2(son[v]);

2 对于v的各个轻儿子u, 显然有 $\text{top}[u] = u$, 并且 $w[u] = \text{totw} + 1$, 进行dfs_2过程, 这就求出了top和w。

→ dfs1: 记录所有的重边

```
void dfs1(int u,int father,int d)
{
    dep[u]=d;fa[u]=father;siz[u]=1;
    for(int i=head[u];i>0;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=father) //不是回向边
        {
            dfs1(v,u,d+1);
            siz[u] += siz[v];
            if(son[u]==0 || siz[v]>siz[son[u]]) son[u]=v;
        }
    }
}
```

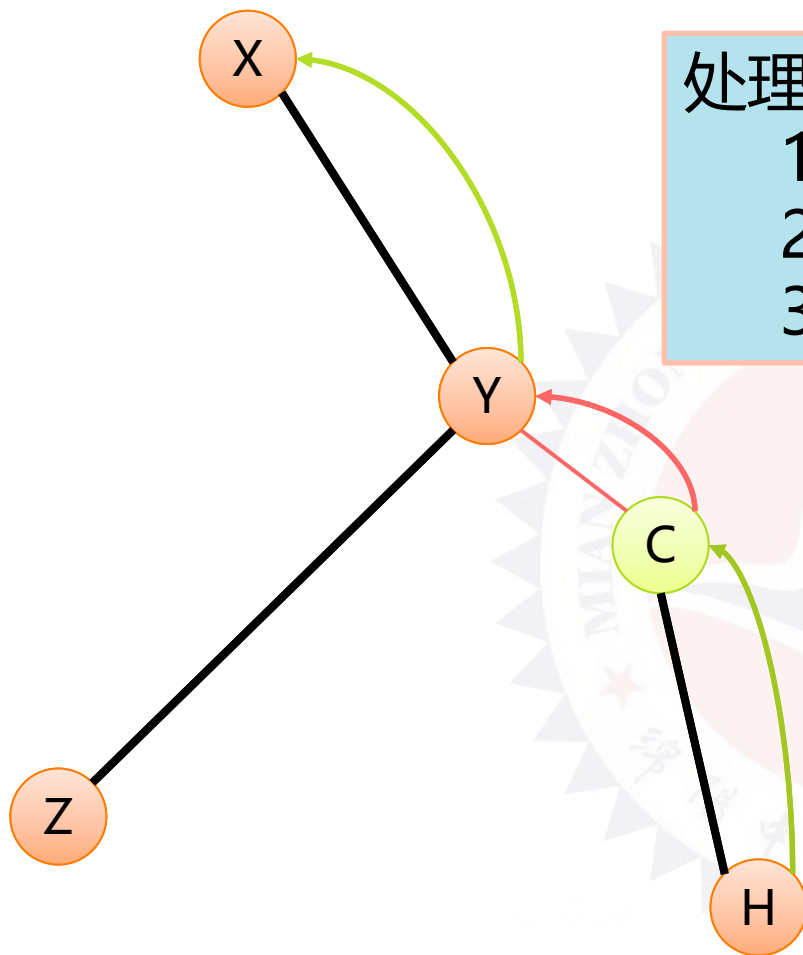


➡ dfs2: 连重边成重链

```
void dfs2(int u,int tp)//tp表示u点所在链的顶端节点
{
    top[u]=tp;
    tid[u]=++tim;
    pos[tid[u]]=u;
    if(son[u]==0) return;
    dfs2(son[u],tp);
    for(int i=head[u];i>0;i=edge[i].to)
    {
        int v=edge[i].to;
        if(v!=son[u] && v!=fa[u])
        {
            dfs2(v,v);
        }
    }
}
```

➔ 线段树的过程，建树

```
void built(int id,int lef,int rig)
{
    tree[id].left=lef;tree[id].right=rig;tree[id].sign=0;
    if(lef==rig)
    {
        tree[id].sum=data[pos[lef]]; return;
    }
    int mid=(lef+rig)>>1;
    built(id*2,lef,mid);
    built(id*2+1,mid+1,rig);
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
}
```



3、处理Y到X

注：黑色的表示重链。

修改操作

如将u到v的路径上每条边的权值都加上某值X

如右图所示，较粗的为重边，较细的为轻边。

节点编号旁边有个红色点的表明该节点是其所在链的顶端节点。

边旁的蓝色数字表示该边在线段树中的位置。

图中1-4-9-13-14为一条重链。

当要修改11到10的路径时。

第一次迭代： $u = 11$, $v = 10$, $f1 = 2$, $f2 = 10$ 。

此时 $\text{dep}[f1] < \text{dep}[f2]$ ，因此修改线段树中的5号点，

$v = 4$, $f2 = 1$;

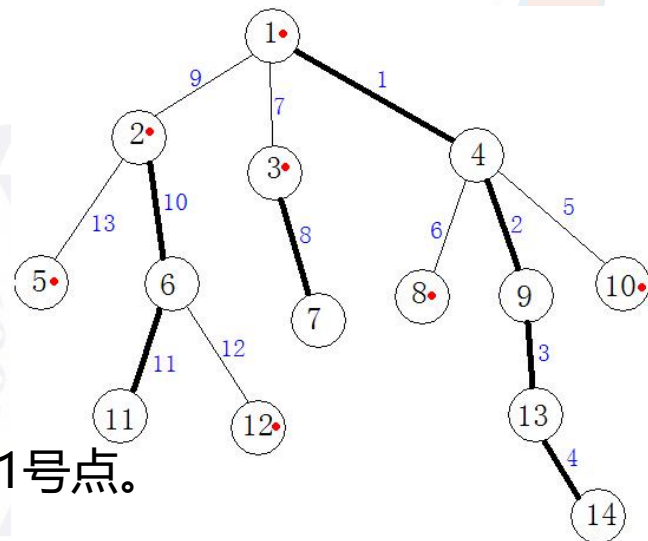
第二次迭代： $\text{dep}[f1] > \text{dep}[f2]$ ，修改线段树中10--11号点。

$u = 2$, $f1 = 2$;

第三次迭代： $\text{dep}[f1] > \text{dep}[f2]$ ，修改线段树中9号点。

$u = 1$, $f1 = 1$;

第四次迭代： $f1 = f2$ 且 $u = v$ ，修改结束。



➔ 修改：u点到v点路径上加上X

```
void change(int u,int v,int val){
    while(top[u]!=top[v])//不在同一条重链上
    {
        if(dep[top[u]]<dep[top[v]]) swap(u,v);
        update(1,tid[top[u]],tid[u],val);
        u=fa[top[u]];
    }
    if(dep[u]>dep[v]) swap(u,v);//在同一条重链上，深度小的
    点在线段树中编号肯定比深度大的编号小
    update(1,tid[u],tid[v],val);
}
```

注：update为线段树的更新函数



➔ 统计、查询

查询和统计与修改类似





练习题目 P3384

题目描述

如题，已知一棵包含 N 个结点的树（连通且无环），每个节点上包含一个数值，需要支持以下操作：

- 操作1：1 x y z 表示将树从 x 到 y 结点最短路径上所有节点的值都加上 z
- 操作2：2 x y 表示求树从 x 到 y 结点最短路径上所有节点的值之和
- 操作3：3 x z 表示将以 x 为根节点的子树内所有节点值都加上 z
- 操作4：4 x 表示求以 x 为根节点的子树内所有节点值之和

数据范围：对于100%的数据： $N \leq 10^5, M \leq 10^5$



习题题目

- P3178[HAOI2015]树上操作
- P3995 树链剖分
- P2590 [ZJOI2008]树的统计
- Hzwer 训练题
- <http://hzwer.com/category/algorithm/graph-theory/tree/tree-chain-partition>



The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two data series is visible on the right side.

Thank you!